# MATRIX CHAIN OPTIMIZER 3D

**Interval Dynamic Programming Visualizer**

Team 09

## Project Description

Matrix Chain Optimizer 3D is a Java-based web application that demonstrates how interval dynamic programming finds an efficient order for multiplying a sequence of matrices. The order of the matrices remains unchanged, but changing their parenthesization can significantly affect the number of scalar multiplications.

The application allows users to configure the number of matrices and enter compatible dimensions. It computes the minimum multiplication cost, derives the optimal parenthesization, compares the result with left-to-right multiplication, and displays the dynamic programming cost table.

## Key Features

- Accepts the number of matrices and their dimensions as input.
- Builds matrix representations from the configured dimensions.
- Calculates minimum scalar multiplication cost using interval dynamic programming.
- Displays the optimal matrix multiplication order (parenthesization).
- Compares optimized cost with naive left-to-right multiplication.
- Reports operations saved and percentage efficiency improvement.
- Displays the DP cost table for matrix intervals.
- Provides a 3D-style visual representation of the matrix chain.
- Includes Play, Pause, Previous, Next, and Reset controls for the visualization.
- Provides a responsive browser interface for learning the optimization process.

## Technologies Used

- Java 17
- Apache Maven
- HTML
- CSS
- JavaScript
- Java built-in HTTP server
- Interval Dynamic Programming

## Algorithms

The project description includes the following four algorithms, with their purpose and standard time complexity:

### 1. Knuth–Morris–Pratt (KMP)

A string-matching algorithm that searches for a pattern in a text by using a Longest Prefix Suffix (LPS) table, avoiding repeated comparisons. Time complexity: O(n + m), where n is the text length and m is the pattern length.

### 2. Rabin–Karp

A string-matching algorithm that uses hash values and a rolling hash to find possible pattern matches. Average time complexity: O(n + m); worst-case time complexity: O(nm), where n is the text length and m is the pattern length.

### 3. Matrix Chain Multiplication (MCM)

An interval dynamic programming algorithm that finds the parenthesization with the minimum number of scalar multiplications. For dimensions p, its recurrence is:

```
dp[i][j] = min over i ≤ k < j of { dp[i][k] + dp[k+1][j] + p[i−1] × p[k] × p[j] }
```

Time complexity: $O(n^3)$  Space complexity: $O(n^2)$

### 4. Edmonds–Karp

A maximum-flow algorithm that uses Breadth-First Search (BFS) to find augmenting paths in a residual network. Time complexity: $O(VE^2)$, where V is the number of vertices and E is the number of edges.

## Expected Outcome

The application demonstrates how dynamic programming can reduce unnecessary scalar multiplications by choosing an efficient parenthesization. It presents numerical cost comparisons, the optimal order, a DP table, and an interactive visual representation to make the optimization process easier to understand.